RAG 项目

解决 API 服务并发瓶颈与资源泄漏工单

八维文化与产业研究院

2026 年 1 月

八维教育 BAWAY EDUCATION

1 基本信息

工单对应的项目名称	RAG
工单编号	人工智能 NLP-RAG-解决 API 服务并发瓶颈与资源泄漏工单
创建时间	2026 年 1 月 23 日
创建人	王洪荣

2 任务描述

任务描述	**任务需求与问题分析** 在完成“工单 2”并对 6 个测试问题实现 100%准确率后，系统进入试运行阶段。当模拟 10 个以上用户同时使用时，系统出现以下故障现象： **响应延迟飙升：/api/v1/chats_openai/{chat_id}/chat/completions** 问答接口的平均响应时间从约 2 秒陡增至 10 秒以上。 **内存泄漏与崩溃**：API 服务器的容器内存使用率随运行时间持续缓慢增长，在连续运行数小时后触发内存限制 (OOM Killer)，导致服务崩溃重启。 **GPU 资源利用不当**：在仅处理文本问答请求（未触发重新解析文档）时，GPU 内存也被大量占用且释放不及时。）。

初步根因推测：

瓶颈一：文档解析与模型加载：每个请求可能都在独立初始化重量级的文档解析器（DeepDoc）或视觉语言模型，导致并发时资源争用和初始化延迟。

瓶颈二：请求处理无队列/限流：API 服务可能未对耗时操作（如复杂检索）进行异步队列处理，导致高并发时请求堆积，线程阻塞。

泄漏点一：全局缓存或会话未释放：解析后的中间对象、模型推理的中间张量或对话会话上下文可能未被垃圾回收机制正确释放。

泄漏点二：数据库/向量库连接池泄漏：处理每个请求时，数据库连接或向量搜索客户端未正确归还至连接池。

具体任务

任务一：搭建压测环境与基准测试

1、使用 docker-compose 部署完整的 RAGFlow 服务（API Server + Task Executor + 向量数据库+ Redis）。

2、使用 jmeter 编写压测脚本，模拟以下两种场景，并记录基

准性能数据:

1）场景 A（高频问答）：模拟 20 个并发用户，持续向 聊天/搜索接口发送已入库文档的简单问题（如工单 2 中问题 1、2）。

2）场景 B（混合负载）：模拟 10 个并发用户，其中 5 个进行问答，5 个同时上传并解析新的 PDF 文档。

任务二：定位性能瓶颈与内存泄漏点

1、应用性能剖析:

1）、使用 py-spy 对 API 服务进程进行采样，找出 CPU 热点函数。

2）使用 memory-profiler 或 objgraph 跟踪典型请求处理周期内的内存对象创建与留存情况。

2、系统资源监控:

1）使用 nvidia-smi 和 docker stats 持续监控 GPU 与容器内存变化。

2）检查 redis-cli 的 INFO stats 命令输出，观察任务队列堆积

情况。

3、代码静态分析：重点审查以下疑似模块的代码：

1）api/service.py 或 web_server/ 目录下的请求处理入口。

2）模型加载与封装的单例管理代码（查找 Model， Pipeline 等类）。

3）文档解析器（DeepDoc）的调用上下文。

任务三：实施优化与修复

1、实现资源池化与单例模式：

1）将文档解析器、重排模型等重量级组件改造为全局单例或进程内缓存，避免重复初始化。

2）确保数据库连接、向量搜索客户端使用正确的连接池配置。

2、优化请求处理链路：

1）对耗时较长的“文档解析”任务，确保其已通过 Redis 队列由 Task Executor 异步处理， API Server 不阻塞。

2）对“检索+生成”链路，考虑引入异步处理或使用 asyncio

	优化 I/O 等待，或使用内存中的请求队列配合工作线程池进行限流。 任务四：验证优化效果 1、使用与任务一相同的压测场景和脚本，进行优化后测试。 2、对比优化前后的关键性能指标（响应时间、吞吐量、内存占用曲线），生成性能测试报告。
工时预估	2 人日
产出物	1、压测脚本与基准报告：包含 jmeter 压测脚本 和第一轮压测结果数据与图表。 2、问题排查分析报告：详细记录性能瓶颈定位过程、内存泄漏点的代码位置及根因分析。 3、优化后的代码：所有修复性能问题和内存泄漏的代码补丁，以及集成了 Prometheus 监控的代码。 4、最终性能测试报告：包含优化后压测数据、与基准的对比分析、以及达到验收标准的结论。 5、部署与运维文档：更新后的 DEPLOYMENT.md，OPERATION.md 或等效文档。 6、演示视频：展示优化前后，在相同压测条件下，系统资源监控面板（如 Grafana）的对比情况，以及服务稳定处理请求的能力。

验收标准	**1、性能稳定性验收（核心）：** ● 场景 A（20 并发问答）：持续运行 10 分钟，P95 响应时间 ≤ 3 秒，且无请求失败。容器内存增长在 10% 以内波动，无持续上涨趋势。 ● 场景 B（10 并发混合负载）：持续运行 10 分钟，服务无崩溃。文档解析任务正常进入队列并被消费，问答请求 P95 响应时间 ≤ 5 秒。 **2、资源泄漏验收**：服务连续稳定运行 12 小时 后，对比启动时的内存占用（RSS），增长不得超过 20%。 **3、文档验收**：产出的部署与运维文档内容准确、步骤清晰，能够指导新的运维人员完成服务部署与日常监控。
备注	1、性能调优是迭代过程，建议使用“假设-验证-修改”的方法。务必在每次修改前后进行可对比的基准测试。 2、强烈建议在 Docker 中配置合理的资源限制（--memory, --cpus），以模拟生产环境约束并提前暴露问题。